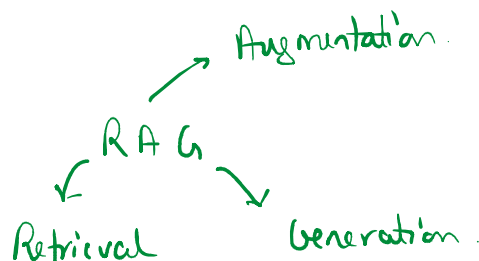


Graph RAG

- ① Traditional RAG v/s Graph RAG
- ② Graphs → Concept
- ③ Ingestion Pipeline.
- ④ Retrieval Pipeline.
- ⑤ Limitations.

Traditional RAG



2 steps

① Data ingestion

Vector Database

Semantic meaning
of text

↓
Vector Embeddings

a) Knowledge source → Load, Parse, text extract

Document loaders

↓
Unified format

b) Split the documents into chunks.



accurate embeddings

→ Text splitters.

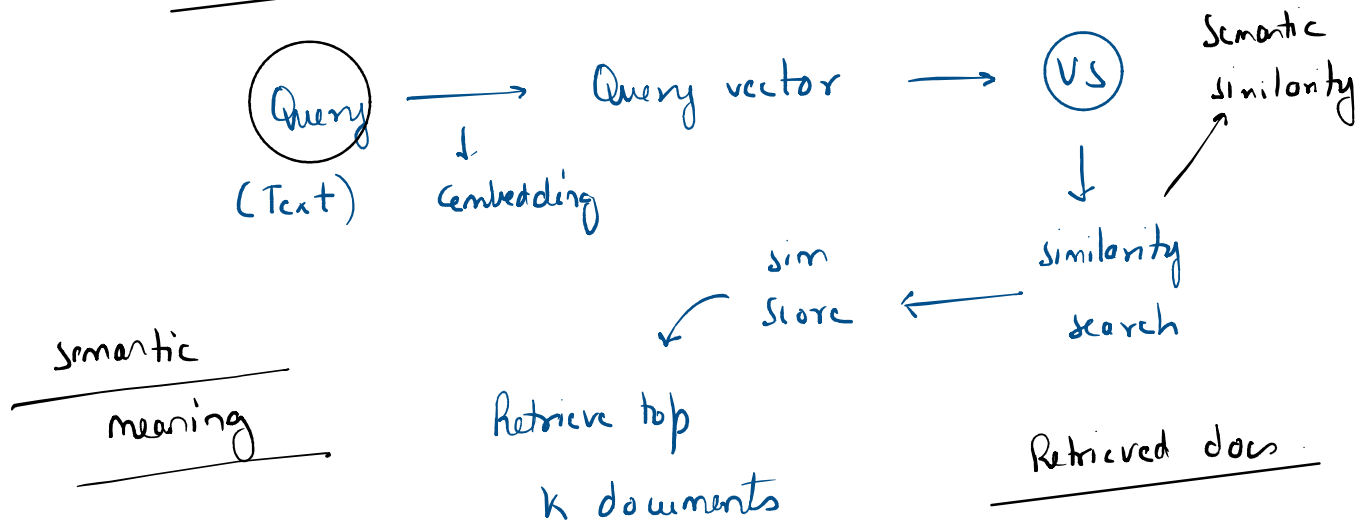
c) Embedding model → Vector embeddings

c) Embedding model $\rightarrow$ Vector embeddings

$\downarrow$ semantic meaning $\rightarrow$ numbers.

d) Vector Databases $\rightarrow$ Storage + Indexing

② Retrieval pipeline



Augmentation



Context + Query $\rightarrow$ LLM $\rightarrow$ Grounded Response

① Foundation $\rightarrow$ Vector Database



Text embeddings.

② Retrieval $\rightarrow$ similarity search $\rightarrow$ semantic meaning of text

Query $\rightarrow$ Relationships, Multi Hop questions Location.

Eg. which company of Elon Musk is HQ in Austin?

Relationship

Chunks $\rightarrow$ embeddings $\rightarrow$ (vs)

Graph RAG

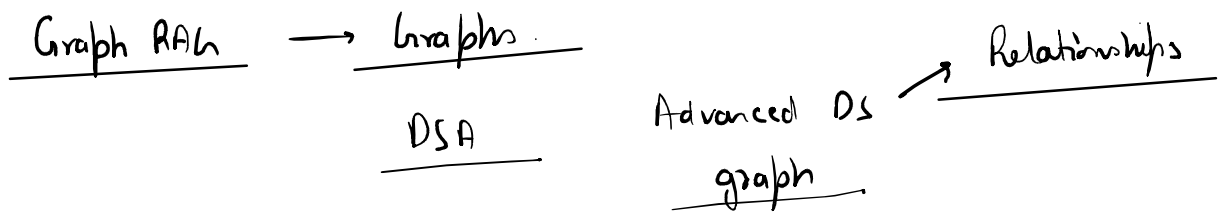
Query + doc - embedding
Scores

Chunk $\rightarrow$ individual entity

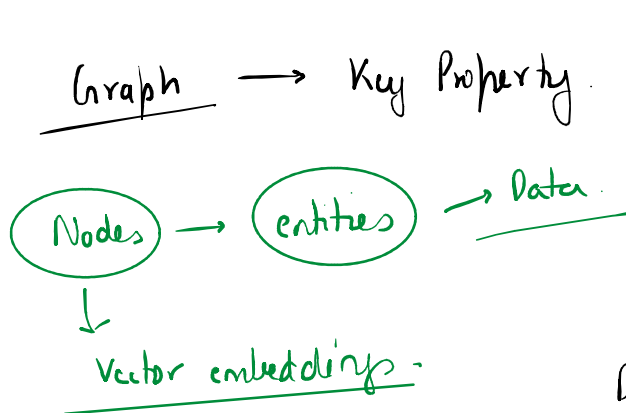
$\downarrow$
embed $\rightarrow$ embeddings

+ $\uparrow$ score
(Query)

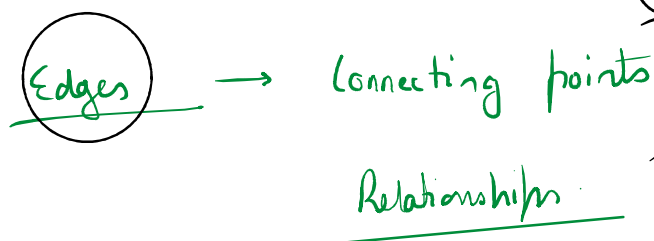
Independent of each other



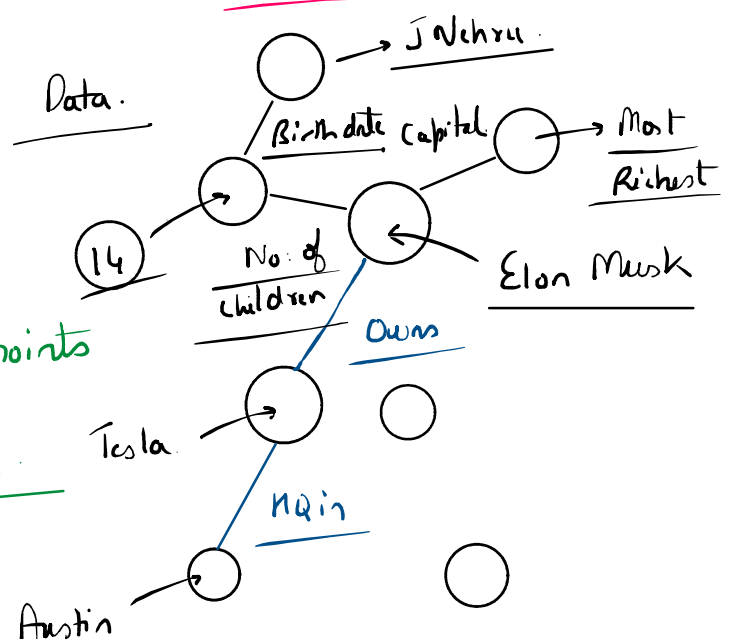
Vector Databases → Vector Embeddings
Relationship ×



completely individual



- 2 Key components
- ① Nodes → Entities
 - ② Edges → Relationships



Graph RAG → Relationships b/w entities

① Nodes → Data.

② Edges → Connection Points (Relationships)

What are Graphs?

Graph RAG

① Nodes → Data

② Edges → Connections

No.

Python Objects

Graph

Data Structure

→ Data Store Container

Order

Components

✓
Nodes

↓
Edges

Python Objects

Name: Aman
Age: 25



is friend of

Name: Mohit
Age: 28



class Edges:

relationship:

start-node:

end-node:

class Node:

id:

data:

datatype:

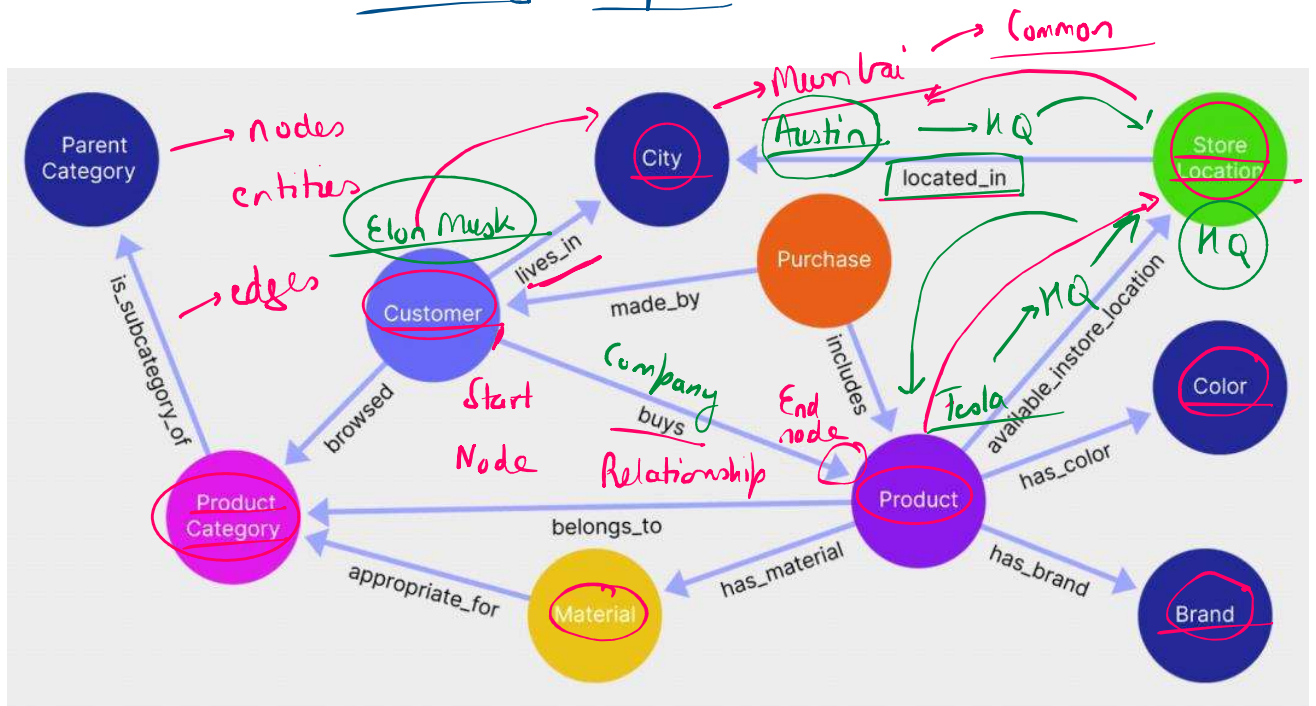
Graph

→ Collection of nodes → entities / Data

(Name, Age)

Collection of edges (Relationships)

Knowledge Graph



Nodes → separate data entities

Edges → Relationships.

Nop → Traverse the graph } → Navigate

Graph traversal.

Steps → Nops

Graph RAG

→ Traverse

Nop

Eg. Which company of Elon Musk is HQ in Austin?

Multi Hop Queries

Vector Databases

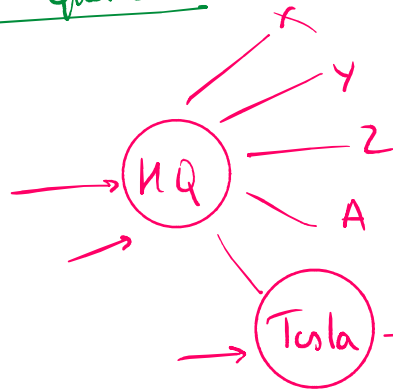
Multi Hop Queries

Start Node

Austin

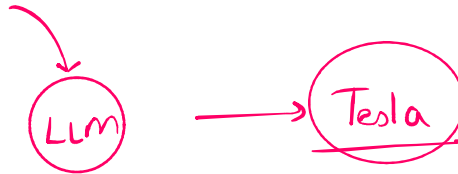


Context



Vector Databases

Relationships



How do Graph RAG works?

Graphs (Relationship) Multi Hop Queries

① Node → Data.

Entity

Chunks → Vector Database

Graph → Graph Database.

② Edges → connections / relations.

Vector Database

Graph database → Data Persist

① Entities

② Connections

① Persist your embeddings

② Indexing

② Data → Retrieval quick.

Elon Musk → Entity Data

text

Node

Data agnostic

```
Node: Elon Musk
- name: "Elon Musk"      ← text property
- type: "Person"         ← entity type
- source: "chunk 42"     ← which chunk it came from
- embedding: [0.23, 0.81, -0.14, ...] ← vector stored ON THE SAME NODE
```

Cypher query language. Readable

Data → Entity

Node → text

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE m.name = 'Top Gun'
RETURN a.name
```

Nodes / edges.

separate

Nodes

embedding model

Graph Database

Persist our data → Entities
→ Connections.

Indexing → Embeddings

Edges

Relationship: Acted in

Neo4J

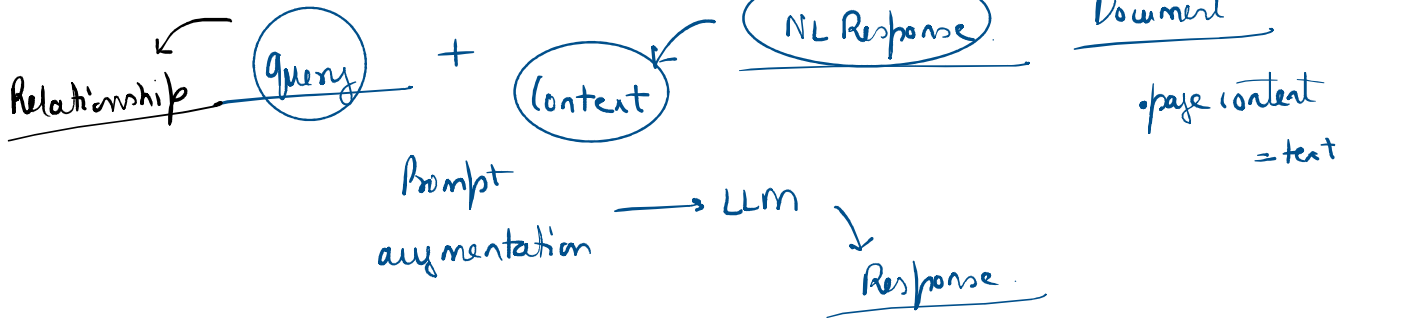
type: Actor name: Tom Cruise

Acted In.
(Movie: Top Gun)

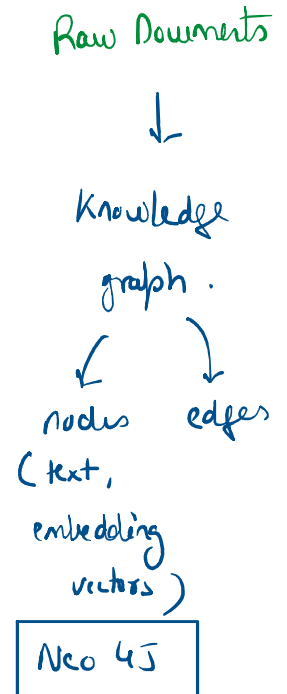
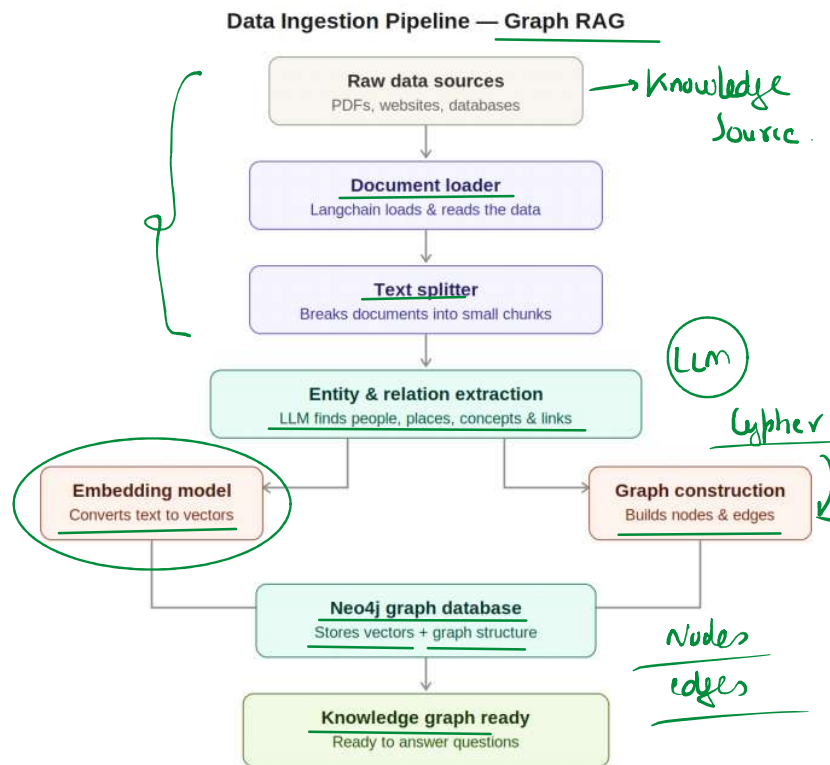
Relationship: Acted in

Start node — Actor →

End Node → Movie →



Ingestion Pipeline



✓ Vector Database

① knowledge source → raw documents
Document loaders → Unified format

→ multiple file types: pdf, web search, doc, csv

② split into chunks → text splitters.

③ embedding model → embedding vector → semantic meaning

④ Vector Database → Embedding vectors → store.

Graph Databases.

Graph Databases.

① Knowledge Source $\rightarrow$ Raw documents $\rightarrow$ text
Document loader.

② split doc into chunks $\rightarrow$ Text splitters $\rightarrow$ text

③ Graph databases

- entities (Nodes) Embeddings X
- ↓
- Relationships (edges)

Chunks $\rightarrow$ entities + Relationships (Extract)

NER → Named entity recognition.
text → entities (type)

Prompts specific

① All the entities (Nodes)

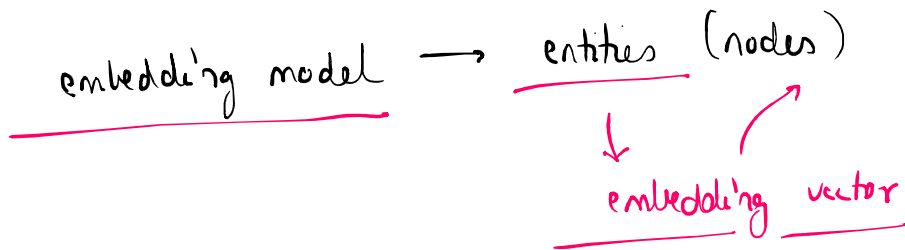
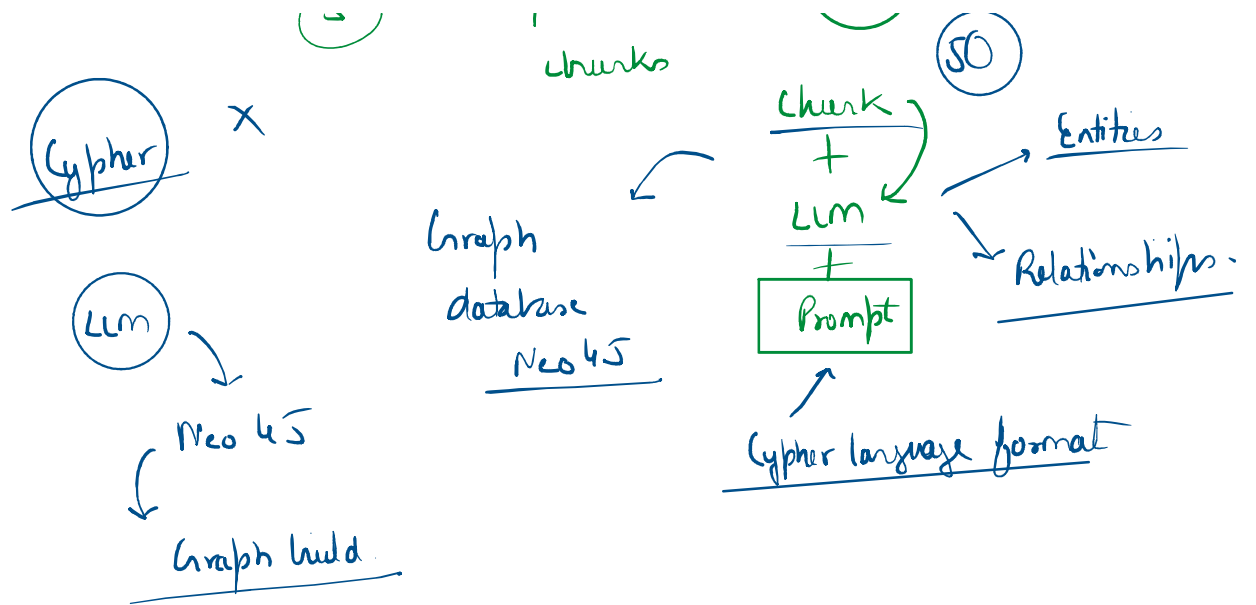
② All the relationships b/w the entities (edges) ↓ ↓

Graph $\rightarrow$ data to be added
 $\downarrow$
extract

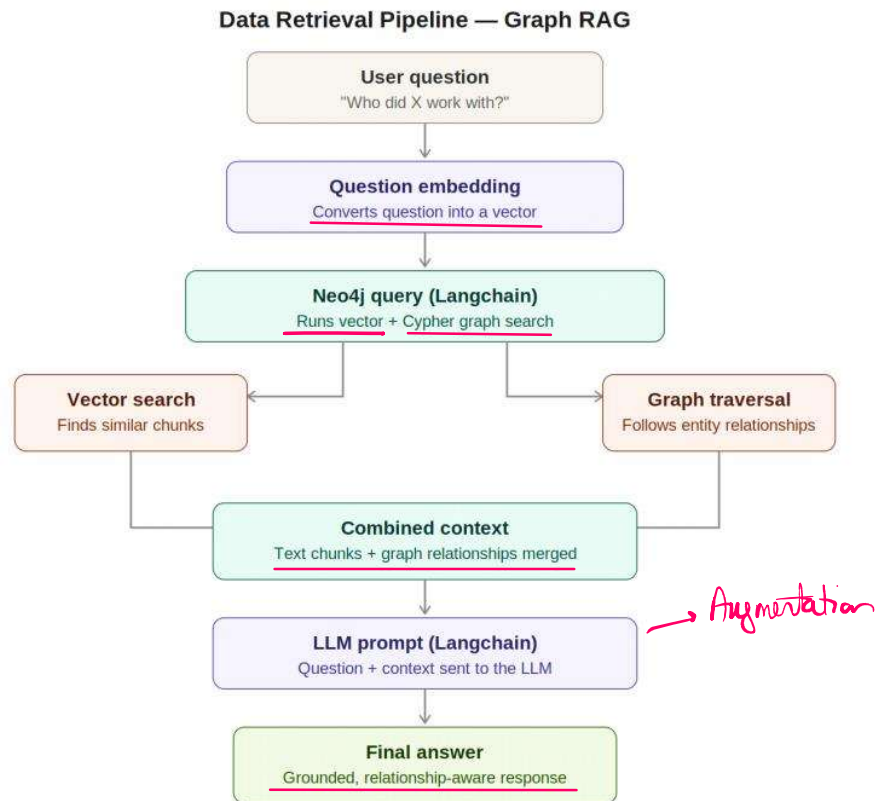
LM $\rightarrow$ Semantic Meaning
Context aware

Pdf document $\rightarrow$ 5 pages

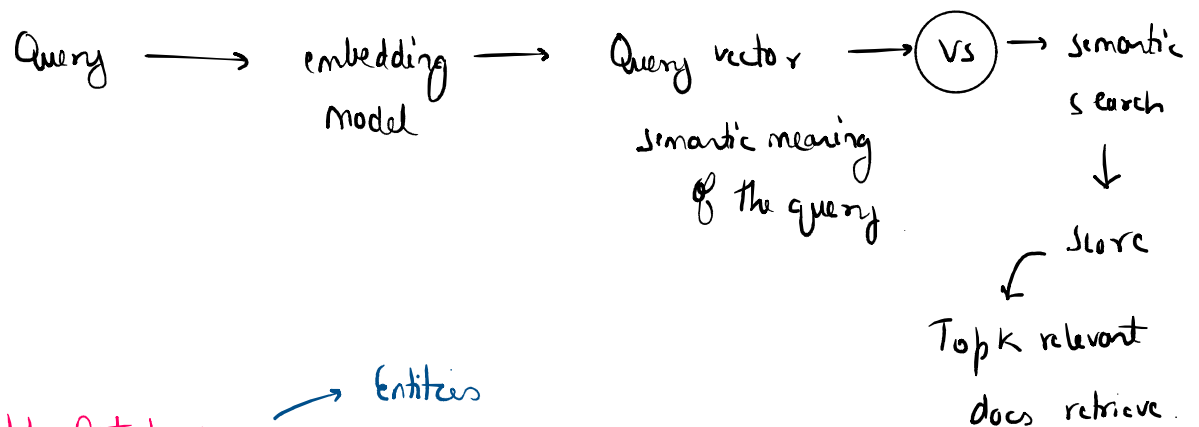
5 → split into chunks → 50



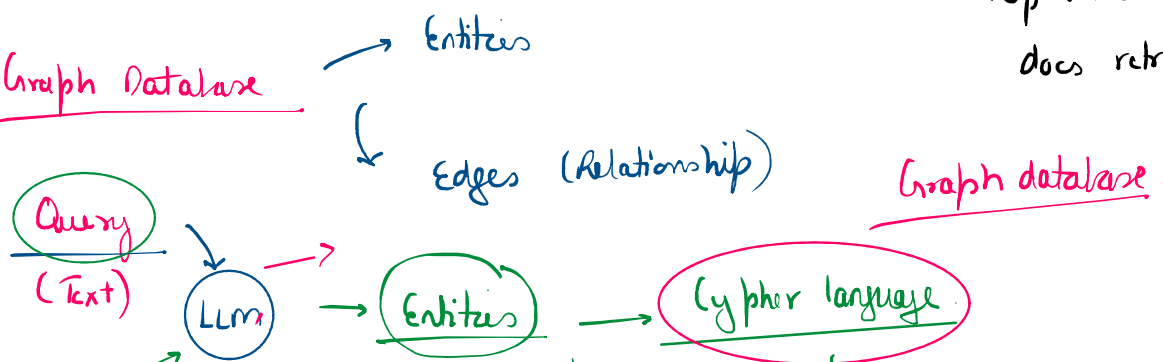
Retrieval Pipeline

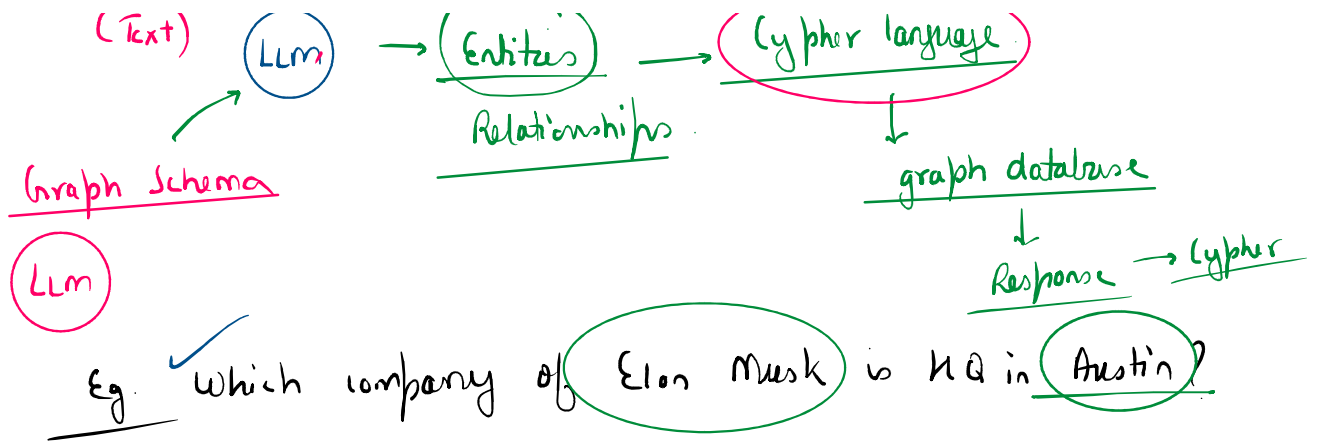


Vector Database



Graph Database





Nodes available:

- Person (properties: name, description)
- Organization (properties: name, description)
- Location (properties: name)

Relationships available:

- FOUNDED (Person → Organization)
- HEADQUARTERED_IN (Organization → Location)
- BORN_IN (Person → Location)

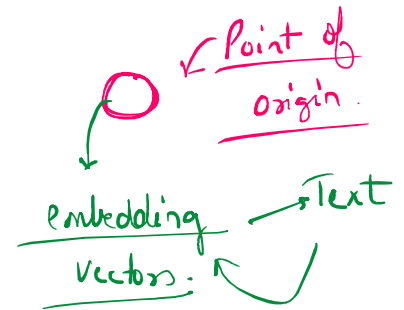
Graph Schema

Neo 4J

Target node

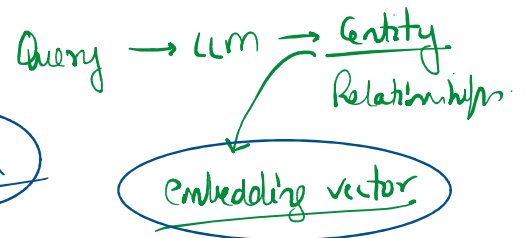
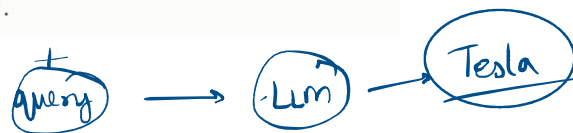
2 hops. → NL context

[Elon Musk] --FOUNDED--> [Tesla] } 1st hop
 [Elon Musk] --FOUNDED--> [SpaceX] }
 [Elon Musk] --FOUNDED--> [xAI] } Coverage?
 [Tesla] --LOCATED_IN--> [Austin] } 2nd hop
 [SpaceX] --LOCATED_IN--> [Hawthorne] }



Context

Elon Musk founded Tesla. Tesla is located in Austin.
 Elon Musk founded SpaceX. SpaceX is located in Hawthorne.
 Elon Musk founded xAI.



User Query (text)

↓

Entity Extraction → "Elon Musk" → embedding vector

↓

Semantic Search → Find [Elon Musk] node in Neo4j

↓

Graph Traversal → Follow edges outward

↓

Context Assembly → Collect nodes + relationships

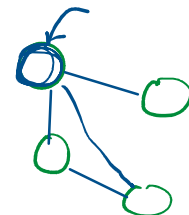
↓

LLM → Generate final answer

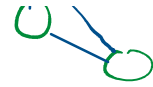
Elon Musk } Embedding model.
 Austin }

Embedding vector

Graph databases

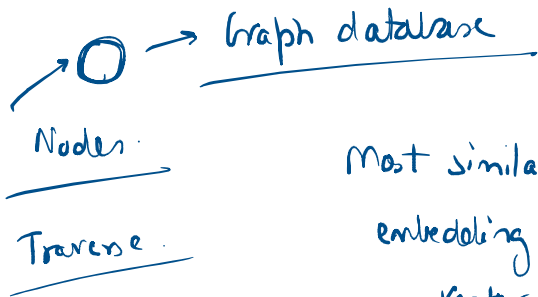
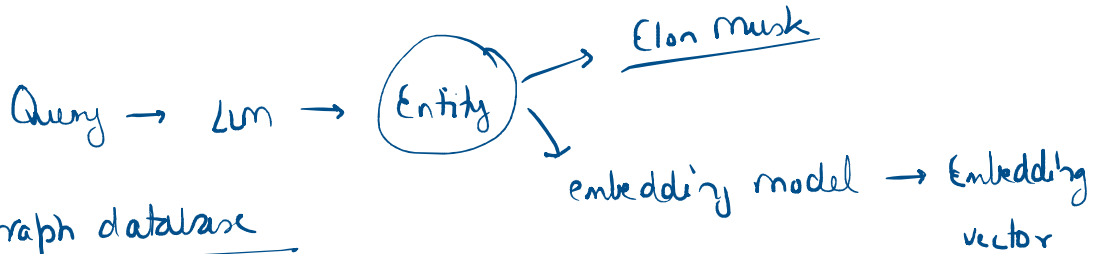
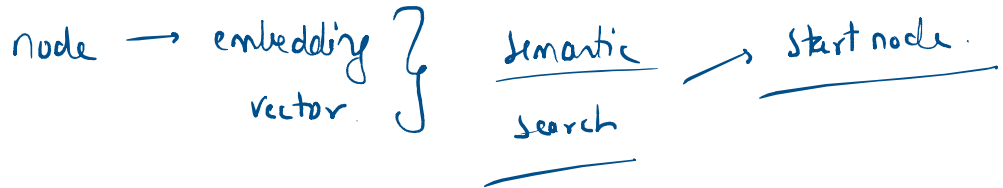


Graph databases -



① Semantic Search → Start node

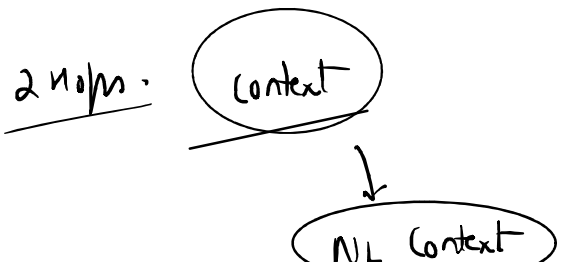
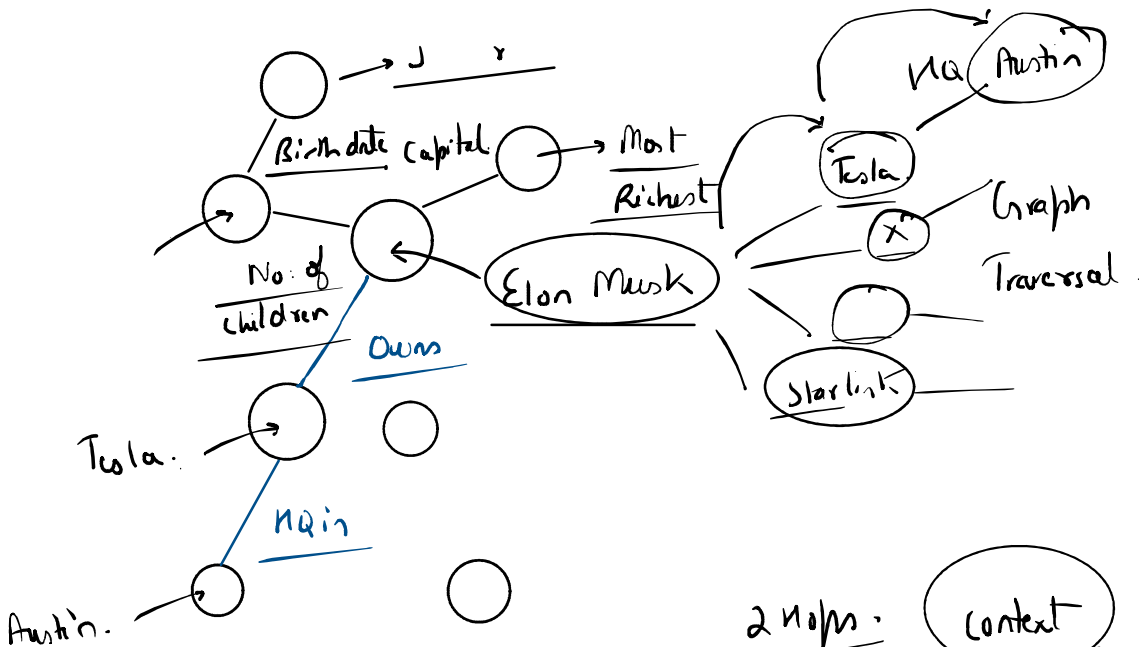
② Graph traversal → Context → target



Most similar
embedding
vectors

Semantic
search

Graph database
100



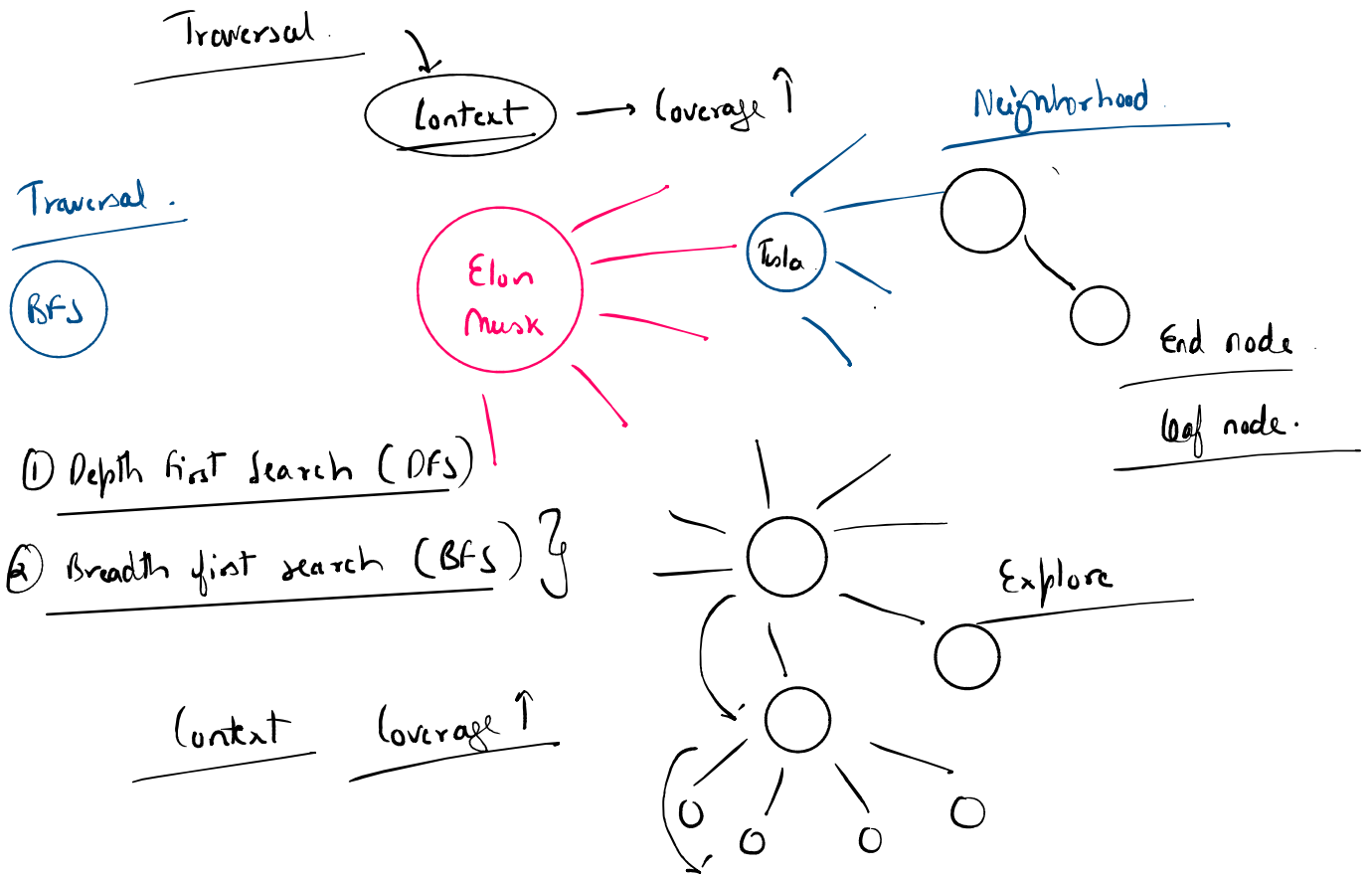
Tweet → Text (Query)

Input → Text (Query)

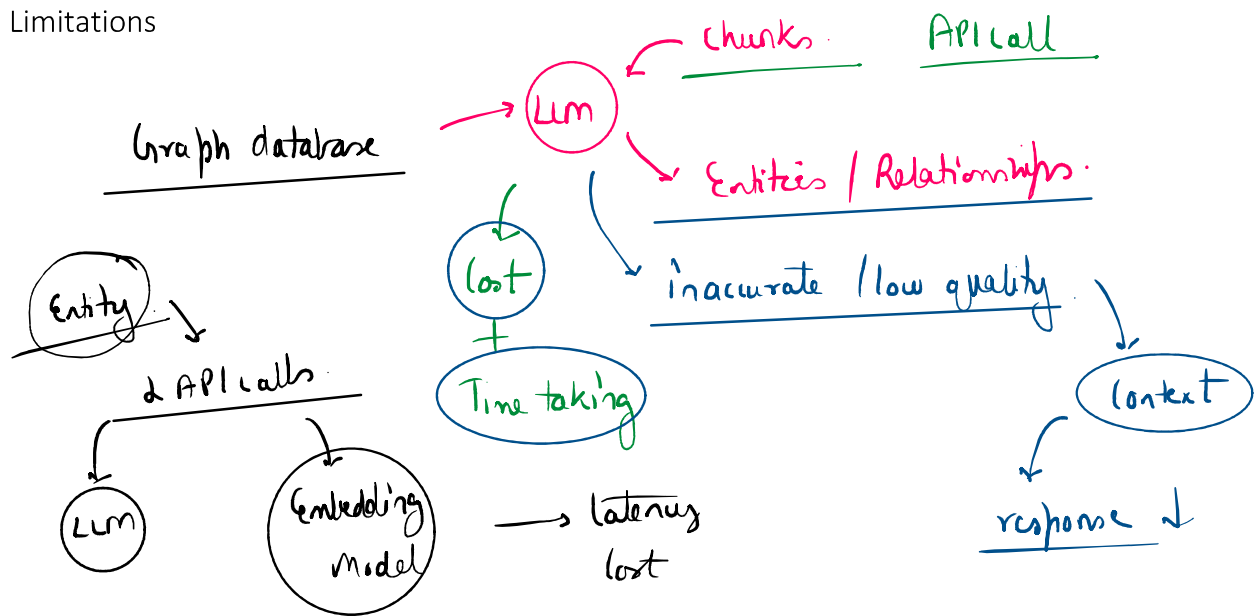
Response → Text

↓
NL Context

Graph database
↓ ↓
Nodes Edges



Limitations



Use Graph RAG when

Data has many interconnected entities

Questions require multi-hop reasoning

Relationships across documents matter

Use Regular RAG when

Simple Q&A over small documents

Semantic similarity is sufficient

Fast, cheap ingestion is a priority

LLM + Embedding

Embedding Model

